

Evidencia de aprendizaje 4

Documentación de la Arquitectura y Modelo de Datos

José David Cardona Mazo

Universidad Digital de Antioquia

Infraestructura y arquitectura para Big Data

CESAR LUIS VASQUEZ

PREICA2602B010136

Medellin, Antioquia

2026

Introducción

En el presente trabajo se buscó el desarrollo de la actividad 4 de la materia de infraestructura y arquitectura para big data.

Aquí vamos a abordar algunos temas puntuales, pero empecemos por entrar en contexto con el objetivo de nuestro proyecto en la actualidad las organizaciones manejan grandes volúmenes de datos que provienen de múltiples fuentes y en formatos muy heterogéneos entonces para que esta información sea útil en la toma de decisiones pues es necesario diseñar e implementar sistemas automatizados conocidos como *pipelines* o tuberías de datos y estos procesos nos permiten extraer la información desde su origen para poder limpiarla, transformarla y por ultimo enriquecerla antes de almacenarla en bases de datos analíticas.

Como lo indicamos anteriormente en el presente documento describiremos la arquitectura y el modelo de datos desarrollados para el Proyecto Integrador, la solución que construimos abordó todo el ciclo de vida completo de los datos sobre un conjunto de productos alimenticios.

A lo largo del proyecto, se integraron tres fases consecutivas (EA1, EA2 y EA3) que van desde la extracción inicial hasta la consolidación de un modelo de datos enriquecido, simulando el funcionamiento de una arquitectura moderna de datos en la nube.

Visión Global de la Arquitectura

Para simular el comportamiento de un entorno Big Data en la nube aquí seguimos el patrón de arquitectura por capas, el pipeline lo estructuramos en tres etapas las cuales son:

1- Ingestión de Datos - EA1 (Capa Bronce):

Esta primera etapa se realizó para extracción masiva de más de 1.000 registros desde la API REST de **OpenFoodFacts** nosotros para en caso de que la API principal falle entonces incluimos en el script un mecanismo de contingencia (**fallback**) hacia la API de **DummyJSON** que fue la que utilizamos y ya los datos se guardan en su formato bruto dentro de una base de datos relacional local en **SQLite (src/db/ingestion.db)** y pues también los colocamos en una base de datos NoSQL documental en la nube (**MongoDB Atlas**). También con esto se generan reportes de trazabilidad en texto (**ingestion.txt**) y tenemos una copia en Excel (**ingestion.xlsx**).

2- Limpieza y Procesamiento - EA2 (Capa Plata):

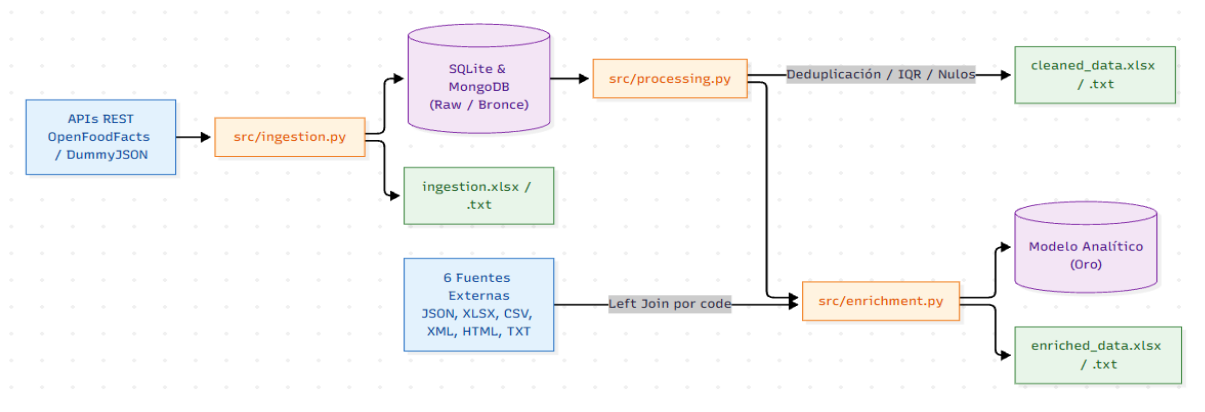
En esta segunda etapa se leen los datos brutos guardados en la base de datos para realizar un control de calidad, aquí se eliminan registros duplicados tomando el código de producto como clave única, se tratan los valores faltantes (nulos) y se corrigen los valores atípicos (outliers) mediante el método del rango intercuartílico (IQR). Finalmente, se aplican técnicas de normalización Min-Max sobre las variables nutricionales numéricas. Como resultado, se generan los archivos **cleaned_data.xlsx** y **cleaning_report.txt**.

3- Enriquecimiento e Integración de los datos - EA3 (Capa Oro):

En la fase final, el dataset limpio se cruza mediante operaciones de combinación sobre la clave code con seis fuentes de datos secundarias en diversos formatos (JSON, XLSX, CSV, XML, HTML y TXT). Esto permite que podamos complementar la

información básica con datos de precios, país de origen, fabricantes, material de empaque, popularidad e impuestos, produciendo el dataset consolidado en **enriched_data.xlsx** y su informe de auditoría **enriched_report.txt**.

Pero veamos esta estructura en un diagrama:



Modelo De Datos

Para organizar la información de todo el proceso de ingesta, limpieza y enriquecimiento, se estructuró un modelo relacional de datos en la base de datos **SQLite** (**src/db/ingestion.db**).

Nuestro modelo se diseñó siguiendo un esquema en estrella adaptado, dividiendo los datos en tres tablas bien definidas para evitar la redundancia y facilitar las consultas analíticas:

1. Tabla Maestra: Esta tabla se almacenan los atributos descriptivos e identificativos del producto obtenidos desde la API REST principal.

dim_productos	
code	TEXT (PK) – Código de barras / ID único
product_name	TEXT – Nombre comercial del producto
brands	TEXT – Marca o fabricante principal
categories	TEXT – Categorías del producto
quantity	TEXT – Contenido neto / Presentación
nutriscore_grade	TEXT – Calificación nutricional (A-E)

2. Tabla de Métricas: Esta tabla se almacenan los datos nutricionales procesados en la EA2, pero sobre estos datos es importante decir que se les aplicó limpieza de nulos también la corrección de valores atípicos (**outliers**) mediante el método del Rango Intercuartílico (**IQR**) y escalamiento Min-Max.

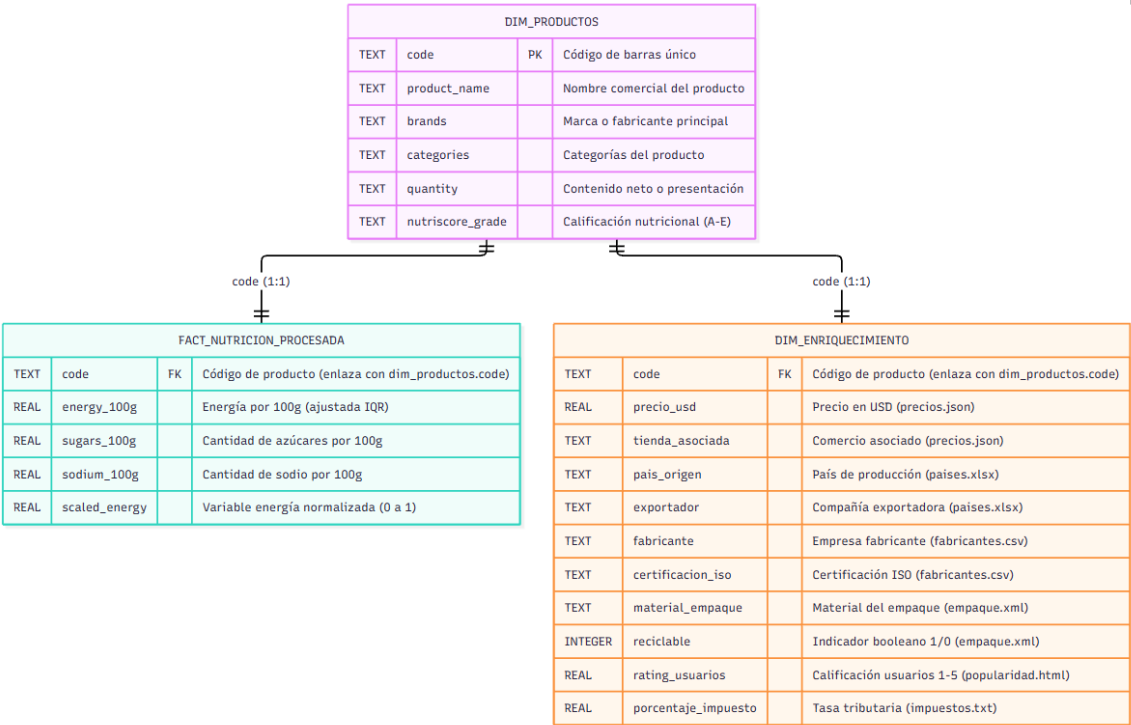
fact_nutricion_procesada	
code	TEXT (FK) – Código enlace dim_productos
energy_100g	REAL – Energía por 100g (ajustada IQR)
sugars_100g	REAL – Cantidad de azúcares por 100g
sodium_100g	REAL – Cantidad de sodio por 100g
scaled_energy	REAL – Energía normalizada (0 a 1)

3. Tabla de Enriquecimiento: En esta tabla ya se consolida toda la información agregada en la EA3 mediante el cruce (**Left Join**) con las 6 fuentes de datos secundarias.

dim_enriquecimiento		
code	TEXT (FK)	Código enlace dim_productos
precio_usd	REAL	Fuente: precios.json
tienda_asociada	TEXT	Fuente: precios.json
pais_origen	TEXT	Fuente: paises.xlsx
exportador	TEXT	Fuente: paises.xlsx
fabricante	TEXT	Fuente: fabricantes.csv
certificacion_iso	TEXT	Fuente: fabricantes.csv
material_empaque	TEXT	Fuente: empaque.xml
reciclable	INTEGER	Fuente: empaque.xml (1/0)
rating_usuarios	REAL	Fuente: popularidad.html (1-5)
porcentaje_impuesto	REAL	Fuente: impuestos.txt

Diagrama del Esquema Lógico de Relaciones

Pero veamos cómo se conectan todas nuestras tablas a nivel de base de datos, la estructura lógica de unión entre estas tres tablas se representa de la siguiente manera:



Justificación del Modelo de Datos

Pero vamos a explicar el porqué de la selección de las herramientas tecnológicas utilizadas en el desarrollo de las tres etapas del proyecto integrador (EA1, EA2 y EA3) primero realizamos una análisis donde encontramos que cada uno contribuye a la escalabilidad, eficiencia y mantenibilidad del Pipeline, también nos permite explicar la simulación del entorno en la nube.

Para esto identificamos y trabajamos sobre tres razones claves, miremos cuales son:

- **Normalización y Eficiencia:** se separa la información en tres tablas (**dim_productos**, **fact_nutricion_procesada** y **dim_enriquecimiento**) esto nos evita tener una tabla "plana" con columnas redundantes o vacías y nos permite la optimización en el almacenamiento y las consultas.
- **Integridad:** Este podemos por ejemplo tomar el código de producto (**code**) como clave primaria (PK) y foránea (FK) los que nos puede garantizar que el enriquecimiento (EA3) no duplique registros y que asocie cada dato al producto correcto.
- **Facilidad de Análisis:** En este punto una vez logrado lo anterior ya se simplifican las consultas SQL con combinaciones (JOINS) datos mas limpias que nos facilitan la conexión con herramientas BI.

3.1 Elección de Herramientas y Tecnologías

- **Python 3.10+:** Lenguaje estándar para ingeniería de datos.
- **Pandas / PySpark:** Pandas para manipulación vectorial rápida en memoria.
- **SQLite3:** Base de datos relacional ligera y sin servidor para estructurar el modelo localmente.

- **MongoDB Atlas:** Almacenamiento NoSQL en la nube para guardar la respuesta JSON bruta de la API sin esquema previo rígido.
- **GitHub Actions:** Orquestación CI/CD que ejecuta secuencialmente los scripts ante cada actualización (push).

Simulación del Entorno Cloud

Este proyecto integra la base de datos generada en las Actividades 1, 2 y 3 con los scripts de procesamiento en dos componentes clave:

1. **Capa de Almacenamiento Simulado (Data Warehouse):**
 - **Capa Bronce (Raw):** Representada por la persistencia NoSQL directa en **MongoDB Atlas** (almacenamiento en la nube) y pues la tabla inicial en **SQLite (src/db/ingestion.db)**, aquí siempre conservando los datos crudos exactamente como los entrega la API.
 - **Capas Plata y Oro (Cleaned / Enriched):** Representadas por las exportaciones tabulares estructuradas en src/xlsx/ y las tablas relacionales finales en SQLite, organizadas para su consumo analítico directo.
 - **Capa de Cómputo Servidor en la Nube (Serverless Processing Pipeline):** Esta se simuló mediante los **ejecutores virtuales en la nube de GitHub Actions**, importante dejar claro que en cada ejecución automatizada, GitHub Actions nos aprovisiona un entorno virtual temporal, nos instala dependencias y nos ejecuta scripts necesarios.

Flujo de Datos y Automatización

Primero queremos recordar que el ciclo de vida de los datos a lo largo del pipeline automatizado se desarrolla a través de tres etapas que son secuenciales y están interconectadas (EA1, EA2 y EA3):

Primera etapa - Ingestión de Datos (EA1)

- **Extracción:** El script ingestion.py se conecta mediante peticiones HTTP GET a la **API REST** de **OpenFoodFacts**, esto se realiza una extracción paginada para obtener un volumen superior a **1.000 registros únicos** de productos de la categoría bebidas.
- **Fallback:** Se configura por si la API principal no responde o presenta tiempo de espera agotado, el script conmuta automáticamente hacia la API de respaldo.
- **Almacenamiento Bruto (Capa Bronce):** Aquí ya los datos extraídos sin transformar se persisten localmente en la base de datos relacional SQLite (**src/db/ingestion.db**) en la tabla ingestión y se sincronizan de forma remota en la base de datos NoSQL MongoDB Atlas.

Para finalizar se crea la muestra tabular **src/xlsx/ingestion.xlsx** y el reporte inicial de auditoría y trazabilidad **src/static/auditoria/ingestion.txt**.

Segunda etapa - Limpieza y Preprocesamiento (EA2)

- **Lectura:** Aquí el script processing.py lee los registros sin transformar almacenados en la base de datos SQLite (**ingestion.db**).
- **Control de Calidad:** Primero se identifican y eliminan los registros duplicados utilizando el código único de barras del producto (code) como clave de deduplicación.
- **Tratamiento de Valores Atípicos:** Aquí ya se aplican algoritmos basados en el método del Rango Intercuartílico (IQR) sobre las variables nutricionales numéricas (energy_100g, sugars_100g, sodium_100g) para recortar valores extremos o erróneos.

- **Imputación y Normalización:** Los valores nulos en columnas numéricas se imputan mediante la media poblacional, mientras que en variables cualitativas se asignan etiquetas por defecto. Posteriormente, se aplican transformaciones de escalamiento Min-Max (0 a 1) sobre la variable energética.

Para finalizar se guarda la estructura limpia en **src/xlsx/cleaned_data.xlsx** y se genera el informe detallado de auditoría de limpieza en

src/static/auditoria/cleaning_report.txt.

Tercera etapa - Enriquecimiento (EA3)

- **Carga de Datos Limpios:** Desarrollamos el script `enrichment.py` toma como insumo el dataset procesado en la EA2.
- **Integración:** Aquí se realiza la lectura e ingesta concurrente de **6 fuentes de datos secundarias** alojadas en la carpeta **src/data_sources/**:
 - `precios.json` (JSON): Datos de precio en USD y tienda comercializadora.
 - `países.xlsx` (Excel): País de origen y empresa exportadora.
 - `fabricantes.csv` (CSV): Nombre del fabricante y certificación de calidad ISO.
 - `empaque.xml` (XML): Material de empaque e indicador de reciclabilidad.
 - `popularidad.html` (HTML): Calificación promedio de los usuarios (rating).
 - `impuestos.txt` (TXT): Porcentaje de impuesto y categoría tributaria.
- **Cruce de Información:** Aquí se realiza una combinación de tablas utilizando el atributo `code` como clave de cruce, garantizando que los datos de mercado se asocien exactamente al producto correspondiente.

Para finalizar se estructura el modelo relacional definitivo y se exportan los resultados enriquecidos a **src/xlsx/enriched_data.xlsx** y el reporte de trazabilidad final en **src/static/auditoria/enriched_report.txt**.

Automatización con GitHub Actions (CI/CD)

Para nosotros poder garantizar que el pipeline sea reproducible, auditable y ejecutable sin intervención manual se realiza la configuración del proyecto y este cuenta con un flujo de trabajo de Integración Continuo (CI/CD) que esta en el archivo

.github/workflows/bigdata.yml.

Funcionamiento del Workflow:

1. **Disparadores:** Primero el flujo se activa automáticamente ante cualquier evento de envío de código (push) a la rama principal (main) o de forma manual mediante el disparador.
2. **Entorno Virtual Cloud:** Seguido del paso anterior se ejecuta sobre un contenedor virtualizado de Ubuntu (ubuntu-latest) con la versión oficial de **Python 3.10**.
3. **Pasos de Ejecución Secuencial:**
 - Descarga la última versión del repositorio.
 - Prepara el intérprete de Python.
 - Instala el proyecto en modo editable y descarga las librerías necesarias.
 - Corre consecutivamente los tres scripts en el orden lógico estricto:
 1. **python src/ingestion.py**
 2. **python src/processing.py**
 3. **python src/enrichment.py**
4. **Gestión:** Una vez finalizada la ejecución anterior entonces GitHub Actions recopila automáticamente todos los informes generados en las carpetas

src/static/auditoria/ y src/xlsx/ y los publica en la sección de **Artefactos** del repositorio para su verificación y descarga

Conclusiones y Recomendaciones

1. **Integración Multifuente y Políglota Exitosa:** El proyecto demostró la viabilidad de diseñar e implementar un *Data Pipeline* automatizado capaz de unificar y consolidar información heterogénea proveniente de más de 7 fuentes de datos en diversos formatos (APIs REST, JSON, XLSX, CSV, XML, HTML y TXT), resolviendo problemas de incompatibilidad de formatos y tipos de datos mediante un atributo clave único (code).
2. **Gobernanza, Trazabilidad y Control de Calidad:** La implementación de reportes de auditoría en cada capa del pipeline (ingestion.txt, cleaning_report.txt y enriched_report.txt), sumada a la aplicación de reglas estadísticas para el tratamiento de valores nulos, *outliers* (mediante IQR) y normalización Min-Max, garantiza que el dataset resultante en la capa *Gold* sea confiable, limpio y esté listo para el modelado o la analítica avanzada.
3. **Arquitectura Modular y Automatizada:** El uso de GitHub Actions como motor de orquestación CI/CD permite ejecutar todo el flujo de ingesta, limpieza y enriquecimiento de forma totalmente desatendida y reproducible. La separación por capas (equivalente a *Bronze*, *Silver* y *Gold*) asegura la mantenibilidad del código y facilita la detección de fallas en cada etapa del procesamiento.

5.2 Recomendaciones para la Implementación en un Entorno Real de Nube

1. **Escalamiento del Almacenamiento a Cloud Object Storage:** En un entorno de producción masivo, se recomienda reemplazar las bases de datos SQLite locales y las carpetas estáticas por contenedores de almacenamiento de objetos administrados

en la nube, como **Amazon S3**, **Google Cloud Storage (GCS)** o **Azure Data Lake Storage (ADLS)**, organizados en subcarpetas estructuradas por capas (bronze/, silver/, gold/).

2. **Migración a Procesamiento Distribuido con PySpark / Databricks:** Si el volumen de datos escala de miles a millones de registros (Terabytes o Petabytes), la manipulación en memoria mediante Pandas debe migrarse a **PySpark** para su ejecución distribuida en clústeres elásticos mediante servicios como **AWS EMR**, **Google Cloud Dataproc** o **Azure Databricks**.
3. **Apache Airflow:** Aunque GitHub Actions cumple y es eficientemente con tareas de CI/CD y pruebas integradas, para un entorno corporativo real se sugiere utilizar orquestadores especializados de flujos de trabajo (*DAGs*) como **Apache Airflow** o **Prefect**, los cuales permiten gestionar dependencias complejas, reintentos automáticos, alertas en tiempo real y monitoreo detallado del rendimiento.

ENLACE A GITHUB

<https://github.com/josedav-17/Infraestructura-y-arquitectura-para-Big-Data.git>